



Servicio Nacional de Aprendizaje
Centro de Biotecnología Agropecuario

Análisis y desarrollo de Software

Ficha: 3409609

Informe de Configuración de Estándares de Codificación en el IDE (PEP8)

Karol Juliana Velasco Rincón y Cristian Fabian Tunaroza Rodriguez

Jose Edwin Gonzales Retamozo

13 de agosto de 2026

Introducción

Breve resumen sobre la importancia de estandarizar el entorno de desarrollo (VS Code) aplicando reglas de calidad, formateo automático y análisis estático para garantizar un código limpio y mantenible.

Objetivos:

- Configurar Visual Studio Code siguiendo las directrices de la norma PEP 8.
- Instalar y parametrizar las herramientas de auditoría estática (*Ruff / Flake8*) y formateo automático (*Black Formatter*).
- Validar la corrección de errores de estilo mediante una prueba práctica en el IDE.

Marco Teórico / Glosario de Conceptos

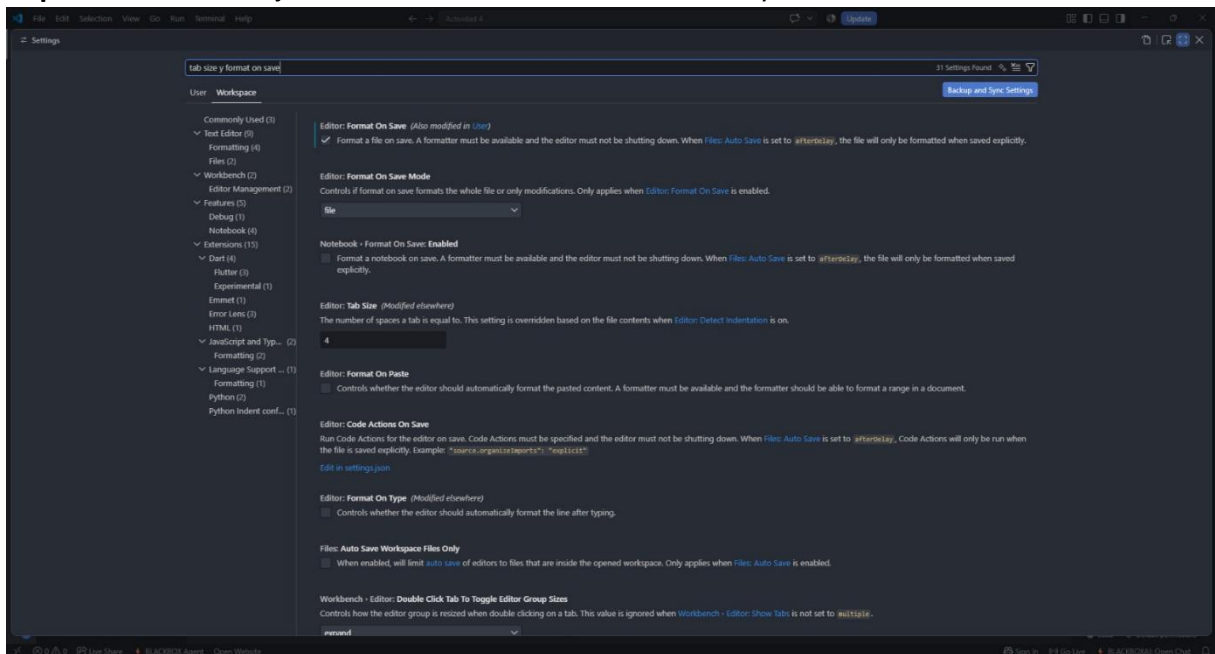
(Definiciones redactadas con tus propias palabras):

- **PEP 8:** Estándar oficial de Python para escribir código limpio, definiendo aspectos como la sangría de 4 espacios, nombrado de variables y clases, y límite de longitud de línea.
- **Linter vs. Auto-formateador:**
 - *Linter:* Herramienta que audita el código en tiempo real para alertar sobre violaciones de estilo y malas prácticas.
 - *Auto-formateador:* Herramienta que reestructura automáticamente la disposición visual del código (espaciados y saltos de línea) al guardar.
- **PascalCase vs. snake_case:**
 - *PascalCase:* Inicia cada palabra con mayúscula (ej. MiClaseSena), obligatorio para clases.
 - *snake_case:* Palabras en minúsculas separadas por guion bajo (ej. numero_ficha), obligatorio para variables y funciones.

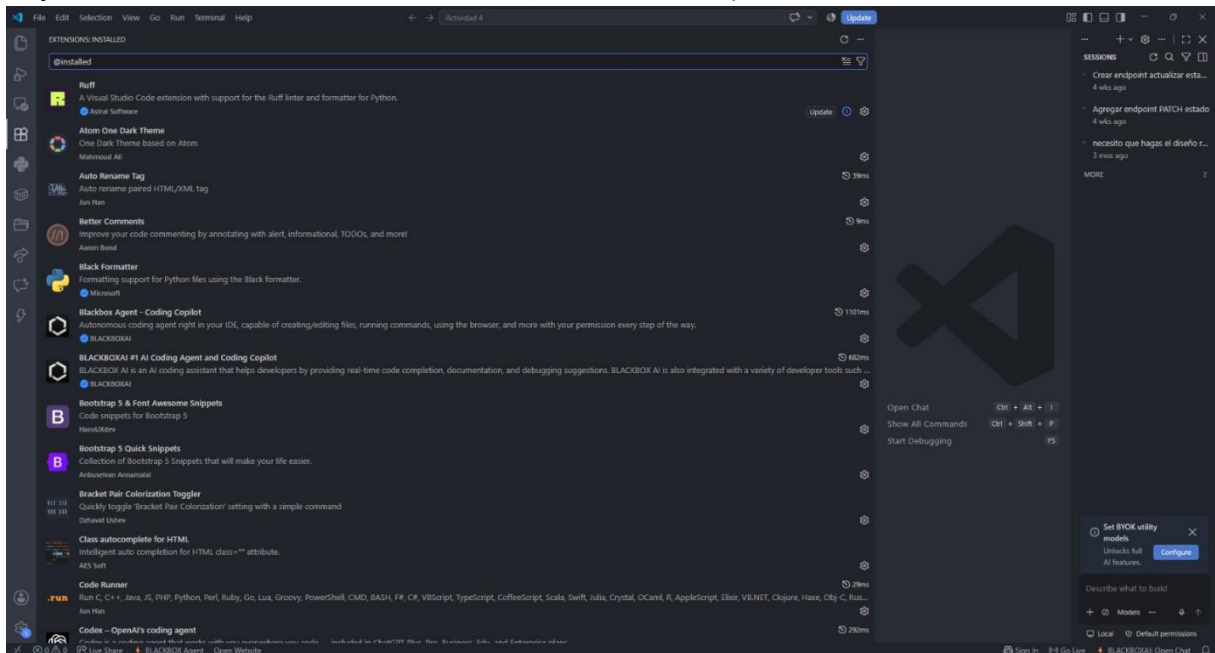


Evidencias Fotografías (Capturas de Pantalla)

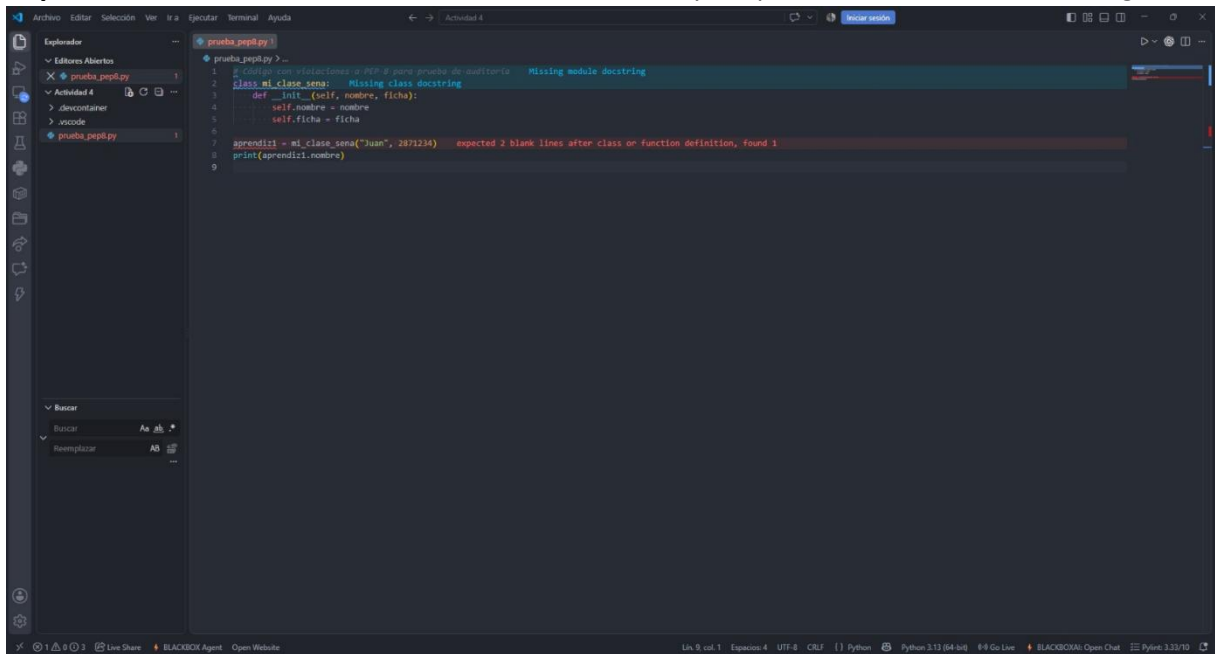
Captura 1: Panel de Ajustes de VS Code mostrando Tab Size: 4 y Format On Save activado.



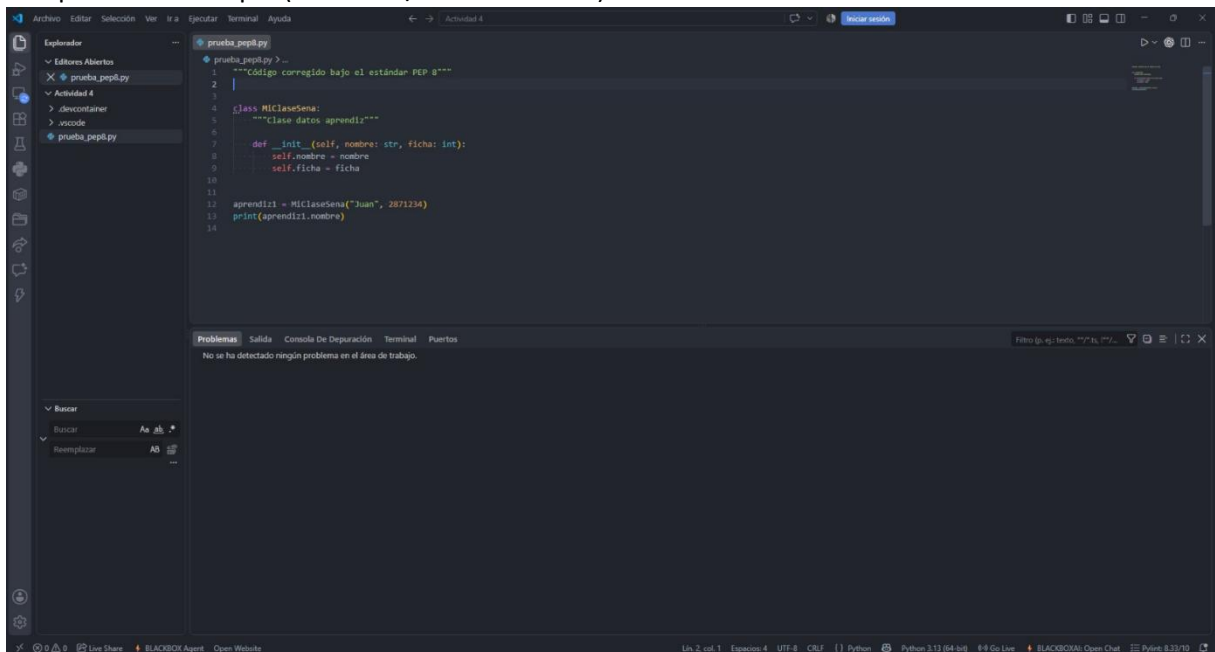
Captura 2: Vista de Extensiones instaladas (Black Formatter y Ruff/Flake8).



Captura 3: Editor mostrando el error de nombrado subrayado por el Linter antes de ser corregido.



Captura 4: Panel de Problemas (Ctrl + Shift + M) y editor mostrando el archivo prueba_pep8.py completamente limpio (0 errores / 0 advertencias).





Código Fuente

```
"""Código corregido bajo el estándar PEP 8"""
```

```
class MiClaseSena:
    """Clase datos aprendiz"""

    def __init__(self, nombre: str, ficha: int):
        self.nombre = nombre
        self.ficha = ficha

aprendiz1 = MiClaseSena("Juan", 2871234)
print(aprendiz1.nombre)
```

Conclusiones Técnicas

Uniformidad y Mantenibilidad: La estandarización mediante PEP 8 garantiza que el código sea legible para cualquier integrante del equipo de desarrollo, facilitando la colaboración y reduciendo el tiempo de revisión técnica.

Prevención de Errores en Tiempo Real: La integración de linters y formateadores automáticos en el IDE ayuda a detectar fallos de estilo y sintaxis de forma temprana, evitando la acumulación de deuda técnica antes de subir cambios al repositorio.